madsql 🤬

Deterministic SQL dialect transcoding from the command line

Matt DeMarco

`oramatt`

Why madsql exists

- SQL migrations are repetitive, brittle, and easy to do inconsistently
- Batch conversion gets harder when scripts contain mixed formatting and multiple statements
- Schema context is often missing when all you have is a query workload
- `madsql` wraps `SQLGlot` and `sqlparse` into one deterministic CLI workflow

What ships today

- `madsql dialects`
  List supported SQL dialect names
- `madsql convert`
  Convert SQL between dialects
- `madsql split-statements`
  Split multi-statement SQL into one file per statement
- `madsql infer-schema`
  Infer creation DDL or JSON from SQL workloads

Supported dialects

Run this to see the current list:

```
madsql dialects
```

Examples from the current repo runtime (32 in total):

`athena`, `bigquery`, `clickhouse`, `duckdb`, `mysql`, `oracle`, `postgres`, `singlestore`, `snowflake`, `spark`, `sqlite`, `trino`, `tsql`

Install and verify

From the repository root:

```
pip install -U pip
pip install -e .
madsql --version
```

Expected runtime shape:

```
madsql 0.11.2
python 3.13.5
sqlglot 29.0.1
sqlparse 0.5.3
sql-metadata 2.20.0
simple-ddl-parser 1.10.0
```

Demo 1

Convert statement from stdin emit to stdout

```
echo 'SELECT TOP 3 [name] FROM dbo.users;' | madsql convert --source tsql --target oracle
```

Output:

```
SELECT "name" FROM dbo.users FETCH FIRST 3 ROWS ONLY
```

Demo 2

Convert a file or directory tree

Single file to stdout:

```
madsql convert --source postgres --target mysql ./input.sql
```

Directory to an output tree:

```
madsql convert --source postgres --target mysql --in ./sql --out ./converted
```

Demo 3

Split multi-statement SQL

```
madsql split-statements \
  --in ./examples/input/mssql/example_queries.sql \
  --out /tmp/split
```

Resulting layout:

```
/tmp/split
├── 20260312-175209-madsql-split-statements-report.md
├── examples
│   └── input
│       └── mssql
│           └── example_queries
│               ├── 0001_stmt.sql
│               ├── 0002_stmt.sql
│               ├── 0003_stmt.sql
│               ├── 0004_stmt.sql
│               ├── 0005_stmt.sql
│               ├── 0006_stmt.sql
│               ├── 0007_stmt.sql
│               ├── 0008_stmt.sql
│               └── 0009_stmt.sql
```

Demo 4

Infer schema from a query workload

```
madsql infer-schema \
  --source singlestore \
  ./examples/input/singlestore/nyc_taxi_queries.sql
```

Sample output:

```
CREATE TABLE nyc_taxi.neighborhoods (
  id BIGINT,
  name TEXT,
  polygon GEOGRAPHY
);

CREATE TABLE nyc_taxi.trips (
  accept_time DOUBLE,
  dropoff_location GEOGRAPHY,
  ...
);
```

Demo 5

Supplement infer-schema with metadata and DDL helpers

```
madsql infer-schema \
  --source postgres \
  --in example_queries.sql \
  --target oracle \
  --out /tmp/artifacts \
  --report \
  --infer-engine hybrid
```

- `SQLGlot` stays primary
- `sql-metadata` supplements query metadata
- `simple-ddl-parser` supplements `CREATE TABLE` extraction
- Hybrid reports are named like `<timestamp>-madsql-infer-schema-hybrid.md`

Demo 6

Infer schema from a query workload for specific target

```
madsql infer-schema \
  --source singlestore \
  --input ./examples/input/singlestore/nyc_taxi_queries.sql \
  --target oracle \
  --create-user \
  --create-user-password 'matt' \
  --pretty
```

Sample output:

```sql
CREATE USER nyc_taxi IDENTIFIED BY "matt";
GRANT CREATE SESSION, CREATE TABLE TO nyc_taxi;

ALTER SESSION SET CURRENT_SCHEMA = nyc_taxi;
```

Sample output:

```sql
CREATE TABLE nyc_taxi.neighborhoods (
  id INT,
  name CLOB,
  polygon SDO_GEOMETRY
);

CREATE TABLE nyc_taxi.trips (
  accept_time NUMBER,
  dropoff_location SDO_GEOMETRY,
  dropoff_time NUMBER,
  num_riders NUMBER,
  pickup_location SDO_GEOMETRY,
  pickup_time NUMBER,
  price NUMBER,
  request_time NUMBER,
  status CLOB
);
```

Demo 7

Infer schema from stdin

```
echo "select x.xxx, y.yyy from foo x, bar y where x.xxx=y.yyy limit 10" | \
madsql infer-schema --target oracle --default-type 'varchar2(100)' --pretty
```

Sample output:

```sql
CREATE TABLE bar (
  yyy VARCHAR2(100)
);

CREATE TABLE foo (
  xxx VARCHAR2(100)
);
```

Demo 8

Infer schema from stdin & derive types

```
echo "select x.xxx + y.yyy from foo x, bar y where x.xxx=y.yyy limit 10" | \
madsql infer-schema --target oracle --default-type 'varchar2(100)' --pretty
```

Sample output:

```sql
CREATE TABLE bar (
  yyy NUMBER
);

CREATE TABLE foo (
  xxx NUMBER
);
```

Demo 9

Infer schema from stdin, create objects, convert & run query

```
echo "use alice; select a.x, b.x from foo a, bar b limit 100" | \
madsql infer-schema --source mysql --target oracle --create-user --create-user-password 'matt' | \
orasql -S matt/matt@localhost/FREEPDB1 | \
sleep 1; echo "use alice; select a.x, b.x from foo a, bar b limit 100" | \
madsql convert --source mysql --target oracle | \
orasql -S alice/matt@localhost/FREEPDB1
```

Output

```
no rows selected
```

Inference details

- `infer-schema` merges evidence from multiple statements
- `CREATE TABLE` contributes explicit types
- `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE VIEW`, and `CREATE INDEX` add table and column references
- `--infer-engine hybrid` keeps `SQLGlot` primary and supplements it with `sql-metadata` and `simple-ddl-parser`
- `convert` and `split-statements` use `--infer-schema-engine hybrid` for schema side artifacts
- Output can be DDL or JSON
- Unqualified columns can be skipped or assigned with low confidence

Observability and error handling

- Default behavior is continue-on-error with a non-zero exit code if failures were recorded
- `--fail-fast` stops on the first failure
- `--ignore-errors` returns exit code `0` while still recording diagnostics
- `--errors errors.json` writes a structured JSON report
- `--log 0` or `--log 1` writes run logs
- `--report` writes a timestamped Markdown summary

Why it works well in CI

- Deterministic outputs for identical inputs and `SQLGlot` version
- Predictable artifact names
- Clear exit codes:
  - `0` for success
  - `1` for completed runs with recorded parse or conversion errors
  - `2` for CLI misuse

Good fits

- Cross-dialect migration prep
- Large SQL cleanup efforts
- Batch conversion for repositories
- Splitting vendor scripts into reviewable units
- Inferring starter DDL from analytic workloads
- CI checks around SQL normalization

Boundaries

- This is a CLI, not a GUI
- It intentionally builds on `SQLGlot`, `sqlparse`, `sql-metadata`, and `simple-ddl-parser`
- Output quality depends on parser support for the source dialect and syntax used
- It is strongest when you want reproducible automation, not hand-tuned one-off rewrites

Recommended first commands

```
madsql dialects
madsql --help
madsql convert --help
madsql split-statements --help
madsql infer-schema --help
```

If the console script is not on your PATH, use:

```
python3 -m madsql <command>
```

Get Started!

Download the binary:
https://github.com/oramatt/madsql/releases

Repository quick start:

```
git clone https://github.com/oramatt/madsql.git
pip install -e .
madsql --version
```

Core idea:

SQL conversion, splitting, and schema inference in one CLI

FAQs

1. What does **madsql** stand for?
   — **madsql** stands for **M**igration **A**ssistant for **D**atabase Dialects and, of course SQL but it could mean something else entirely 🤷‍♂️.
2. Is another SQL translator *really* needed?
   — Honestly, no but I wanted something I could use across multiple projects that supported the workflows I use.
3. Is this *just* a wrapper for SQLGlot and sqlparse?
   — Somewhat but regardless, I don't hide the libraries used.
4. Why preserve relative paths in `--out`/`--output`?
   — Preserving relative paths prevents filename collisions, keeps source-to-output mapping easy to trace, and guarantees deterministic output layout for batch runs and CI diffs.
5. Will you implement a GUI?
   — There is no plan on supporting any other interface besides the terminal because that's my preferred interface.

Questions?

Thank you